

AI / ML Interview Concepts & Questions

1. What is an LLM?

LLM stands for Large Language Model.

It is a type of AI model trained on huge amounts of text data to understand and generate human-like text. These models learn patterns in language and can perform tasks like answering questions, summarizing, translating, coding, etc.

LLM → Large Language Model

AI → Artificial Intelligence

Model → A system trained to learn patterns

Generate → To produce new content

Exam Tip

Start with a clear definition, add key points, use examples if possible, and keep it concise.

Important Fact: GPT-4 is a leading LLM developed by OpenAI.

2. What is a token?

A token is the smallest unit of text that a model can understand. It can be a word, subword, or even a single character.

Example: "unbelievable" → "un", "believ", "able"

3. What is tokenization?

Tokenization is the process of converting text into tokens so that the model can process it.

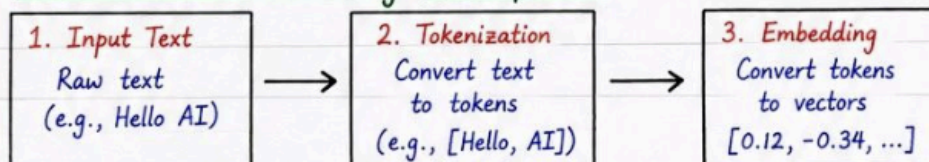
Common Tokenizers: BPE, WordPiece, SentencePiece

4. What is an embedding?

An embedding is a numerical representation of text (or other data) in a continuous vector space. It helps the model understand semantic meaning.

Example: The words "king" and "queen" will have vectors that are close to each other.

Tokenization to Embedding (3-Step Flow)



AI / ML Interview Concepts & Questions

5. What is a Transformer?

A Transformer is a deep learning model architecture introduced in the paper "Attention Is All You Need" by Vaswani et al., 2017.

It is based entirely on attention mechanisms, without using RNNs or CNNs, and is highly effective for sequence-to-sequence tasks like language modeling, translation, summarization, etc.

Key Components: Multi-Head Attention, Feed-Forward Network, Add & Norm (Residual + LayerNorm), Positional Encoding.

Exam Tip

Always mention the key idea + one important component or advantage.

6. Why did Transformers largely replace RNNs/LSTMs for LLMs?

Transformers replaced RNNs/LSTMs because:

- They use **self-attention**, which captures long-range dependencies better.
- They are **parallelizable**, much faster to train on large datasets.
- They handle **long sequences** more efficiently.
- Better **scalability** and performance at scale.

Important Fact:

Transformers enabled the training of much larger models (BERT, GPT, T5, PaLM, etc.) that power modern LLMs.

7. What is self-attention?

Self-attention is a mechanism that allows a token in a sequence to attend to all other tokens in the sequence to compute its representation. It helps the model understand the context and relationships between all words, regardless of their distance.



Exam Tip

Use a simple example or diagram to explain self-attention.

8. What are Query, Key, and Value in attention?

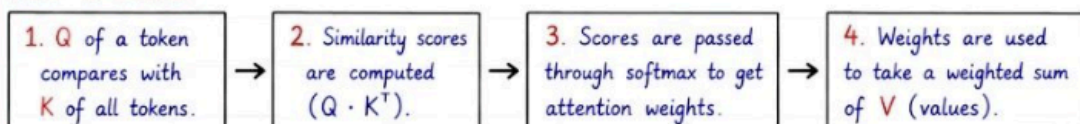
In attention, each token is converted into three vectors:

Query (Q). → What the token is looking for.

Key (K). → What the token contains (used for matching).

Value (V). → The actual information to be aggregated.

How it works:



AI / ML Interview Concepts & Questions

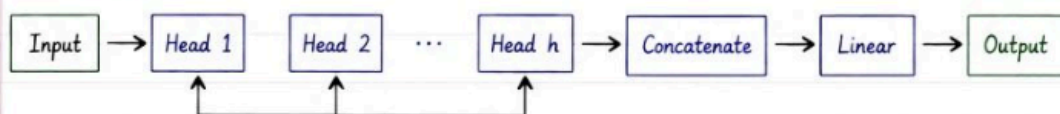
9. What is multi-head attention?

Multi-head attention runs multiple self-attention operations in parallel with different learned projections.

This allows the model to focus on different parts of the input and capture different types of relationships simultaneously.

Why it helps: It improves the model's ability to understand complex patterns and dependencies.

How it works:



Exam Tip

Think of each head as looking at the sequence from a different perspective. Together, they give a richer understanding.

10. What is positional encoding?

Positional encoding is a way to add information about the position of tokens in the sequence, since Transformers do not have any inherent notion of order like RNNs.

It is added to the input embeddings.

Common methods:

1. Sinusoidal positional encoding (fixed)
2. Learned positional encoding (trainable).

Important Fact:

Ashish Vaswani et al. introduced the Transformer in the paper "Attention Is All You Need" (2017).

11. What is the difference between encoder-only, decoder-only, and encoder-decoder Transformers?

Encoder-Only	Decoder-Only	Encoder-Decoder
• Used for understanding tasks like classification, NER, search, embeddings. • Example: BERT	• Used for text generation tasks. • Generates text one token at a time. • Example: GPT	• Used for sequence-to-sequence tasks like translation, summarization. • Example: T5
Input → Encoder → Output (Representation)	Input → Decoder → Output (Generated Text)	Input → Encoder → Decoder → Output ↓ (Generated Text)

Exam Tip

BERT reads the whole sentence.
GPT writes one word at a time.
T5 does both!

Key Difference: Where attention is applied and how the model uses context to produce output.

AI / ML Interview Concepts & Questions

12. What is a context window?

The context window is the maximum number of tokens an LLM can consider in one go (both input and generated tokens). It defines how much text the model can "see" at a time.

Example: A model with a 8K context window can process up to 8,192 tokens in a single request.

Exam Tip

A larger context window allows the model to handle longer documents and maintain more history in conversations.

datamasterclub

13. What happens when input exceeds the context window?

If the input is longer than the model's context window:

- The model will truncate (cut off) some part of the input.
- Important information at the beginning may be lost.
- This can lead to incomplete understanding or wrong answers.

Solution: Use techniques like chunking, summarization, or Retrieval-Augmented Generation (RAG) to handle long inputs.

Important Fact:

Most models do not throw an error; they simply ignore the tokens beyond the window.

14. What are model parameters?

Model parameters are the learnable weights and biases in a model that are updated during training.

They store the knowledge learned from data.

Example: In a Transformer, parameters exist in attention layers, feed-forward layers, embeddings, etc.

Important Fact:

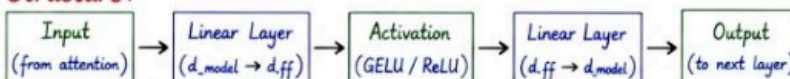
More parameters usually mean more capacity, but also more compute and memory.

15. What is a feed-forward network inside a Transformer?

A Feed-Forward Network (FFN) is a fully connected neural network applied to each position separately and identically.

It consists of two linear layers with a non-linear activation in between (usually GELU or ReLU).

Structure:



Why it's used: Adds non-linearity and helps the model learn complex patterns beyond attention.

Exam Tip

Think of FFN as the "brain's thinking step after paying attention.

AI / ML Interview Concepts & Questions

16. What are residual connections?

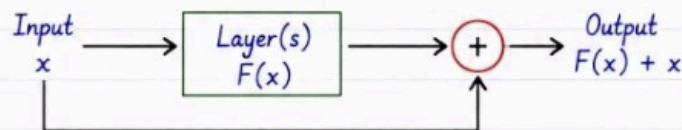
Residual connections (or skip connections) allow the input of a layer (or a block of layers) to be added to its output.

They help in training very deep models by reducing the vanishing gradient problem and improving information flow.

How it works: The input is "skipped" over one or more layers and added to the output.

Formula: $\text{Output} = F(x) + x$

where $F(x)$ is the output of the layers, and x is the input.



Exam Tip

Residual connections make it easier to train deeper networks and often lead to better performance.

17. What is normalization, such as LayerNorm/RMSNorm?

Normalization techniques stabilize and speed up training by normalizing the activations.

- **LayerNorm:** Normalizes across features within a single sample (i.e., across the hidden dimension). Common in Transformers.
- **RMSNorm:** A variant of LayerNorm that normalizes by the root mean square of activations, without subtracting the mean. It is computationally a bit simpler and often used in LLMs.

Why used?

They reduce internal covariate shift and make training faster, more stable, and less sensitive to initialization.

Important Fact:

LayerNorm uses both mean and variance.

RMSNorm uses only the RMS (no mean subtraction).

18. What is the softmax function?

Softmax converts a vector of real numbers (logits) into a probability distribution where all probabilities are between 0 and 1 and sum to 1.

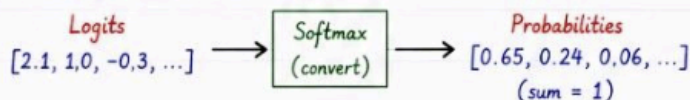
Formula:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

where:

- z_i = i -th logit
- n = total number of logits
- e = Euler's number (≈ 2.718)

Use: In LLMs, softmax is used over the vocabulary to convert logits into probabilities for the next token.



Exam Tip

Softmax amplifies larger values and suppresses smaller ones, turning scores (logits) into probabilities.

AI / ML Interview Concepts & Questions

19. What are logits?

Logits are the raw, unnormalized scores output by the model before applying softmax.

They can be any real numbers (positive or negative).

Softmax converts these logits into probabilities between 0 and 1.

Example: For a vocabulary of 4 tokens, the model might output logits [2.1, 0.5, -1.3, 3.0].

After softmax → probabilities like [0.24, 0.11, 0.03, 0.62].



Exam Tip

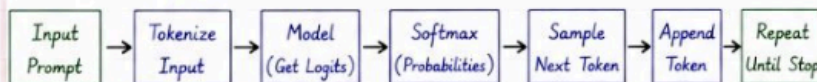
Think: logits are model's confidence scores before normalization. Softmax turns them into probabilities.

20. How does an LLM generate the next token?

LLMs generate text one token at a time in an autoregressive manner.

Steps:

- ① Take the input prompt and tokenize it.
- ② The model processes the tokens and produces logits for the next token.
- ③ Apply softmax to get probabilities over the vocabulary.
- ④ Select the next token (e.g., using greedy, top-k, or top-p sampling).
- ⑤ Append the selected token to the sequence.
- ⑥ Repeat steps ②-⑤ until a stop condition is met (e.g., <eos> token or max length).



Exam Tip

LLMs are autoregressive: they predict the next token based only on previous tokens. One step at a time!

21. What is pretraining?

datamasterclub

Pretraining is the initial phase where a model is trained on a very large amount of unlabeled text data to learn general language patterns and representations.

The objective is usually to predict the next token (causal language modeling) or to fill in missing tokens (masked language modeling).

Example: GPT series uses causal language modeling.

BERT uses masked language modeling.

Goal: Learn a broad understanding of language, facts, syntax, semantics, and world knowledge.

Typical Pretraining Pipeline:



Important Fact:

OpenAI GPT-3

was pretrained on ~45 TB of text data from books, websites, and other sources.

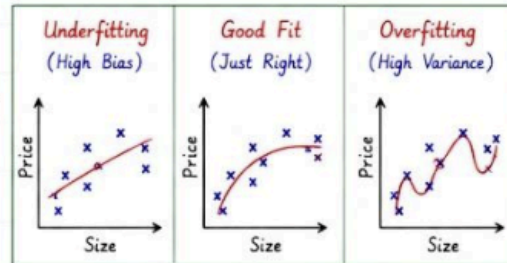
AI / ML Interview Concepts & Questions

22. What is overfitting and how can we prevent it?

Overfitting happens when a model learns the training data too well, including its noise, and performs poorly on unseen data.

How to prevent:

- Get more training data.
- Use regularization (L1, L2).
- Apply dropout.
- Use early stopping.
- Keep the model simple.
- Use cross-validation.



Exam Tip

High training accuracy but low validation/test accuracy is a classic sign of overfitting.

23. What is the difference between bias and variance?

- **Bias:** Error due to overly simple assumptions in the model.
High bias → underfitting.
- **Variance:** Error due to sensitivity to small fluctuations in the training data. High variance → overfitting.

Analogy: Bias is from wrong assumptions, variance is from overreacting to the data.

Important Fact:

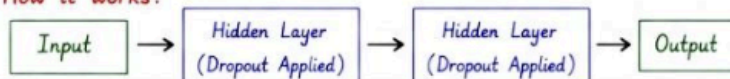
The best models achieve the right balance: low bias and low variance. This is the goal!

24. What is dropout?

datamasterclub

Dropout is a regularization technique where, during training, random neurons are "dropped" (set to 0) with a probability p . This prevents co-adaptation of neurons and reduces overfitting.

How it works:



Used only during training. During inference, all neurons are used with scaled weights.

Exam Tip

Typical dropout rate p is between 0.1 and 0.5.

Don't forget to disable dropout during inference!

25. What is a learning rate?

The learning rate (α) controls how much the model's weights are updated during training.

Update rule: $w_{\text{new}} = w_{\text{old}} - \alpha \times \text{gradient}$

Choosing the right learning rate:



Tips: Use learning rate schedules, warmup, or adaptive optimizers (Adam, RMSprop) for better results.

Important Fact:

Most optimizers (SGD, Adam, etc.) use the learning rate to control weight updates. It's one of the most important hyperparameters!

AI/ML Interview Concepts & Questions

26. What is label smoothing and why is it used?

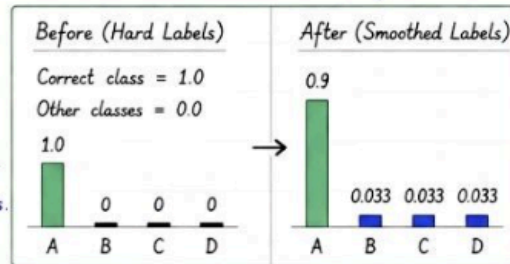
Label smoothing prevents the model from becoming overconfident by softening the target labels. Instead of using 1 for the correct class and 0 for others, we use a small value ϵ for all classes.

Formula:
$$y_{\text{smooth}}(i) = \begin{cases} 1 - \epsilon & \text{if } i \text{ is the correct class} \\ \epsilon / (K - 1) & \text{otherwise} \end{cases}$$

where K = number of classes and ϵ is a small value (e.g., 0.1).

Why used?:

- Reduces overconfidence.
- Improves generalization.
- Helps when labels are noisy.
- Often increases model robustness.



Exam Tip

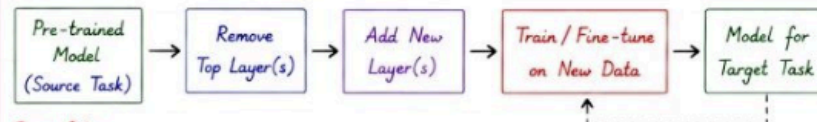
Label smoothing makes the model less confident and more open-minded, which usually leads to better results on unseen data.

27. What is transfer learning? datamasterclub

Transfer learning is a technique where a model learned on one task (source task) is reused as the starting point for another task (target task).

How it works:

- Use a pre-trained model (e.g., trained on ImageNet).
- Remove the final layer(s) of that model.
- Add new layer(s) for your specific task.
- Fine-tune the model on your own dataset.



Benefits:

Saves time, less data needed, better performance, and lower computation.

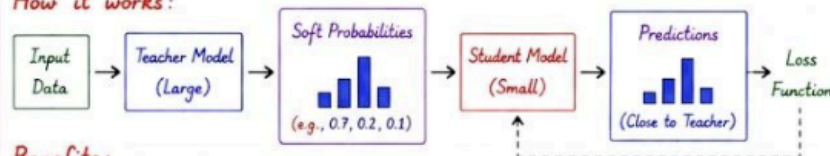
Important Fact:

Transfer learning is widely used in NLP (e.g., BERT), CV (e.g., ResNet), and speech models. It leverages knowledge learned from large datasets.

28. What is knowledge distillation?

Knowledge distillation is a technique where a small student model learns to imitate a larger teacher model. The student tries to match the soft probabilities (outputs) of the teacher instead of hard labels.

How it works:



Benefits:

- Student model becomes smaller and faster.
- Retains most of the performance of the teacher.
- Useful for deployment on devices with limited resources.

Exam Tip

Distillation is like a teacher passing on their knowledge to a student in a simpler form. It's great for model compression!

AI/ML Interview Concepts & Questions

29. What is batch normalization?

Batch Normalization is a technique used to normalize the inputs of each layer in a mini-batch. It helps stabilize and speed up training.

How it works:

It normalizes the activations to have zero mean and unit variance, then scales and shifts them using learnable parameters.

Formula:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

where:

- x = input activation
- μ_B = batch mean
- σ_B^2 = batch variance
- γ, β = learnable parameters
- ϵ = small constant (e.g., $1e-5$)

Benefits:

- Reduces internal covariate shift.
- Allows higher learning rates.
- Acts as a regularizer.
- Helps training converge faster.

Exam Tip

Batch Normalization is applied after the linear layer and before the activation function (e.g., ReLU). It is widely used in deep networks like CNNs.

30. What is gradient clipping?

Gradient clipping is a technique used to prevent the exploding gradient problem in RNNs and other deep networks.

Why used? When gradients become too large, they can cause unstable updates and make the model diverge.

How it works: If the gradient norm exceeds a threshold, it is scaled down.

Formula:

$$\text{If } \|g\| > c, \text{ then } g_{\text{clipped}} = g \times \frac{c}{\|g\|}$$

where g = gradient, c = clipping threshold

Example: If gradient norm = 25 and threshold $c = 5$, then gradients are scaled down by $\frac{5}{25} = 0.2$.

Benefits:

- Prevents exploding gradients.
- Stabilizes training.
- Commonly used in RNNs and LSTMs.

Exam Tip

Gradient clipping does not change the direction of the gradient, only its magnitude. It helps in training deep recurrent models.

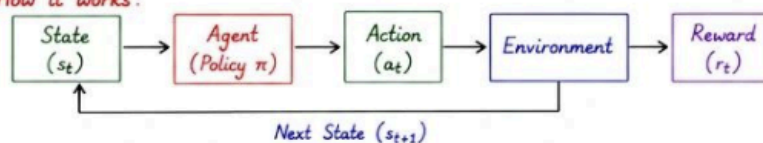
31. What is reinforcement learning?

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative rewards.

Key components:

- **Agent:** Learns and takes actions.
- **Environment:** The world the agent interacts with.
- **State:** Current situation of the environment.
- **Action:** Choices available to the agent.
- **Reward:** Feedback received after taking an action.

How it works:



Exam Tip

RL is all about trial and error! The agent learns the best policy (mapping from state to action) to maximize total reward over time.

32. What is a confusion matrix?

A confusion matrix is a table used to evaluate the performance of a classification model. It compares predicted labels with actual labels.

For Binary Classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Key Metrics:

- Accuracy = $(TP + TN) / (TP + TN + FP + FN)$
- Precision = $TP / (TP + FP)$
- Recall (Sensitivity) = $TP / (TP + FN)$
- F1 Score = $2 \times \frac{\text{Precision} \times \text{Recall}}{(\text{Precision} + \text{Recall})}$

Why used?

- Gives detailed insight into model performance.
- Useful when classes are imbalanced.
- Helps to choose the right metric based on problem type.

Exam Tip

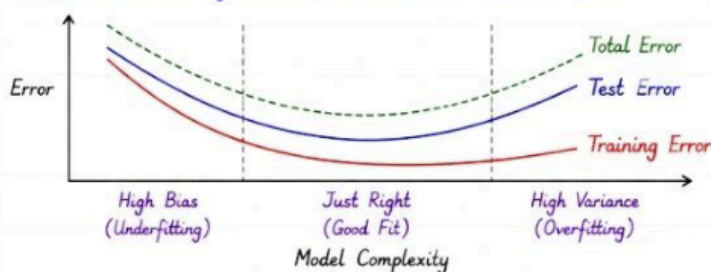
Confusion matrix helps you understand where your model is making mistakes. Always know TP, TN, FP, FN clearly!

33. What is the bias-variance tradeoff?

The bias-variance tradeoff is the balance between two types of errors that prevent a model from generalizing well.

- **High Bias:** Model is too simple. It underfits the data.
Leads to high training error and high test error.
- **High Variance:** Model is too complex. It overfits the data.
Leads to low training error but high test error.

Goal: Find the right balance to minimize total error.



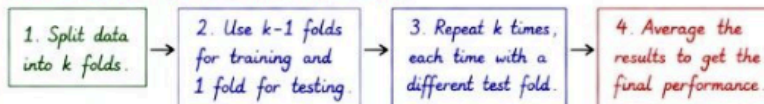
Exam Tip

Simple models → high bias.
Complex models → high variance.
Best models are in the "just right" middle zone!

34. What is cross-validation?

Cross-validation is a technique used to evaluate a model by splitting the data into multiple folds and training/testing the model multiple times.

How it works? (k-Fold Cross-Validation)



Benefits:

- Provides a more reliable estimate of model performance.
- Uses data efficiently for both training and testing.
- Helps in reducing variance in evaluation.
- Common choices: $k = 5$ or $k = 10$.

Exam Tip

Cross-validation gives a better idea of how the model will perform on unseen data compared to a single train/test split.

35. What is the difference between classification and regression?

Both are supervised learning tasks but differ in the type of output they predict.

	Classification	Regression
Output Type	Discrete labels / categories	Continuous values
Example	Spam detection (Spam/Not Spam)	House price prediction (\$)
Algorithms	Logistic Regression, SVM, Decision Trees, Naive Bayes	Linear Regression, Ridge, Lasso, Decision Trees
Evaluation Metrics	Accuracy, Precision, Recall, F1 Score, ROC-AUC	MAE, MSE, RMSE, R^2
Output Range	Finite number of classes	Any real number

Key Idea: Classification answers "What class?" while regression answers "How much?"

Exam Tip

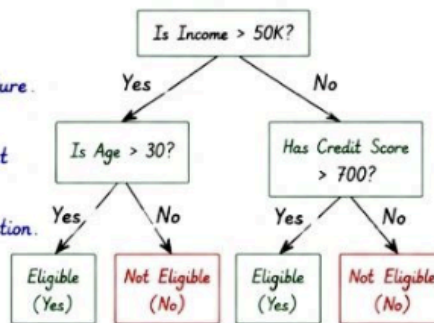
If the target variable is a category → it's classification.
If it's a number (or amount) → it's regression.

36. What is a decision tree? datamasterclub

A decision tree is a flowchart-like structure where internal nodes represent features, branches represent decision rules, and leaf nodes represent output.

How it works:

- Start at the root node (all data).
- Split the data based on the best feature.
- Repeat the split for each branch.
- Stop when a stopping condition is met (e.g., max depth, min samples).
- The final nodes (leaves) give the prediction.



Key Points:

- Easy to interpret and visualize.
- Can handle both numerical and categorical data.
- Prone to overfitting if not pruned properly.

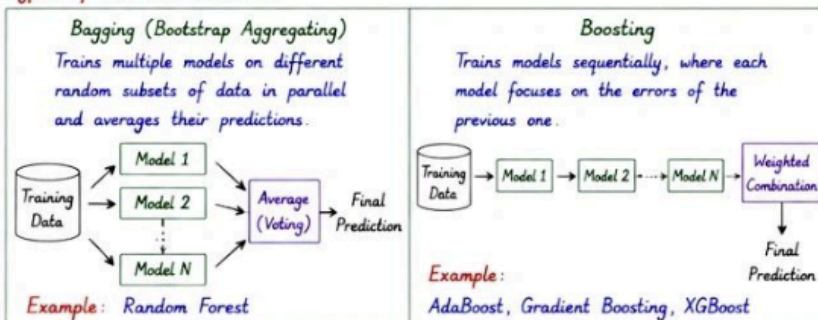
Exam Tip

Decision trees are simple to understand but can overfit. Pruning and setting limits on depth help improve generalization.

37. What is ensemble learning?

Ensemble learning combines multiple models to get better performance than any single model.

Types of Ensemble Methods:



Example: Random Forest

Example:

AdaBoost, Gradient Boosting, XGBoost

- Benefits:**
- Improves accuracy and robustness.
 - Reduces overfitting.
 - More stable predictions.

Exam Tip

Ensemble = multiple models working together.
Bagging reduces variance.
Boosting reduces bias.

38. What is regularization and why is it needed?

Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function based on the model complexity.

Common Types:

- L1 Regularization (Lasso): Adds the absolute value of weights.
- L2 Regularization (Ridge): Adds the square of weights.

Why needed?

- Helps prevent overfitting.
- Improves generalization to unseen data.
- Encourages simpler models and reduces variance.

Formulas:

- L1 Regularization (Lasso):

$$J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n |\theta_i|$$

- L2 Regularization (Ridge):

$$J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n \theta_i^2$$

where:

- λ = regularization strength (hyperparameter)
- θ_i = model weights
- n = number of weights

Exam Tip

Small $\lambda \rightarrow$ less regularization (more complex model).

Large $\lambda \rightarrow$ more regularization (simpler model).

39. What is the bias-variance decomposition?

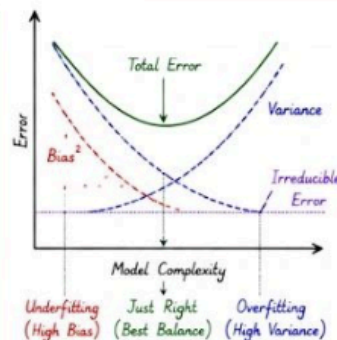
Bias-variance decomposition breaks down the expected generalization error of a model into three parts: Bias, Variance, and Irreducible Error.

Formula:

$$E[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}^2[\hat{f}(x)]}_{\text{Bias (Error from wrong assumptions)}} + \underbrace{\text{Variance}[\hat{f}(x)]}_{\text{Variance (Error from sensitivity to training data)}} + \underbrace{\sigma^2}_{\text{Irreducible Error (Error from noise in the data)}}$$

Interpretation:

- High Bias: Model is too simple $\rightarrow$ underfitting.
- High Variance: Model is too complex $\rightarrow$ overfitting.
- Best model: Achieves the right balance $\rightarrow$ minimizes total error.



Exam Tip

Bias deals with model assumptions.

Variance deals with model sensitivity.

Irreducible error is the noise we cannot remove.

40. What is a support vector machine (SVM)?

SVM is a supervised learning algorithm used for classification and regression. It works by finding the optimal hyperplane that best separates the classes with the maximum margin.

Key Ideas:

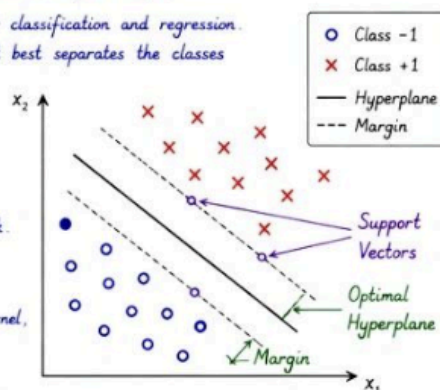
- Finds the hyperplane with maximum margin.
- Uses only the support vectors (closest points) to the hyperplane.
- Can handle non-linear data using the kernel trick.

Types:

- Linear SVM: For linearly separable data.
- Kernel SVM: For non-linear data (e.g., RBF kernel, polynomial kernel).

Margin:

The distance between the hyperplane and the nearest data points of any class.



Benefits:

- Works well in high-dimensional spaces.
- Effective with clear margin of separation.
- Memory efficient (uses only support vectors).

Applications:

Text classification, image recognition, spam detection, bioinformatics, etc.

Exam Tip

SVM focuses on the boundary, not the individual points. More margin $\rightarrow$ better generalization.

41. What is k-nearest neighbors (KNN)?

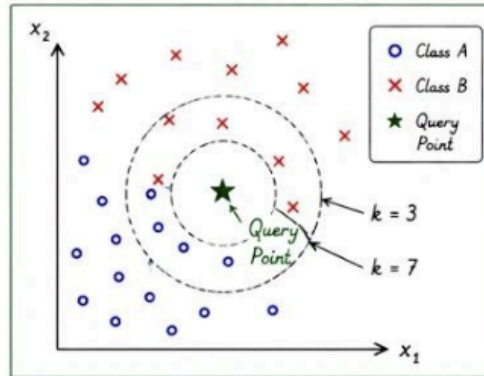
KNN is a simple, instance-based learning algorithm used for classification and regression. It makes predictions based on the "k" closest training examples.

How it works:

- ① Choose the value of k.
- ② Calculate the distance from the query point to all training points.
- ③ Select the k nearest neighbors.
- ④ For classification: use majority vote of neighbors.
For regression: use average (or weighted average) of neighbors.

Key Points:

- Choice of k is important.
- Sensitive to feature scaling.
- Works well for small to medium datasets.
- Uses a distance metric (e.g., Euclidean distance).



Exam Tip

KNN is lazy learning (no training phase). At prediction time, it looks at the training data.

42. What is naive Bayes?

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem with the assumption that features are conditionally independent.

Bayes' Theorem:

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

where:

- C = class
- X = features
- $P(C|X)$ = posterior
- $P(X|C)$ = likelihood
- P(C) = prior
- P(X) = evidence

How it works:

- Compute prior P(C) for each class.
- Compute likelihood P(X|C) assuming feature independence.
- Use Bayes' theorem to compute posterior.
- Predict the class with highest posterior probability.

Benefits:

- Fast and easy to implement.
- Works well with high-dimensional data and small datasets.
- Widely used in text classification (spam filtering, sentiment analysis).

Types:

- Gaussian Naive Bayes (for continuous data)
- Multinomial Naive Bayes (for discrete count data)
- Bernoulli Naive Bayes (for binary data)

Exam Tip

"Naive" means it assumes features are independent, which is often not true in real world → but it still works well!

43. What is principal component analysis (PCA)?

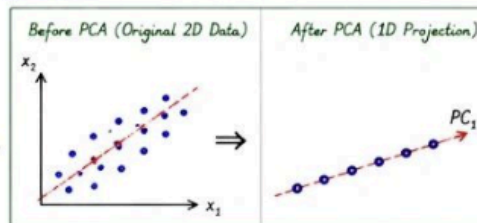
PCA is a dimensionality reduction technique that transforms data into a new coordinate system where the greatest variance lies on the first principal component, the second greatest on the second, and so on.

Steps:

- ① Standardize the data.
- ② Compute the covariance matrix.
- ③ Compute eigenvalues and eigenvectors.
- ④ Sort eigenvectors by descending eigenvalues.
- ⑤ Choose top k eigenvectors.
- ⑥ Project the data onto the new subspace.

Benefits:

- Reduces dimensionality while retaining most of the information (variance).
- Removes noise and redundancy.
- Improves visualization and speeds up downstream models.



Exam Tip

PCA is unsupervised. It preserves variance, not class labels. Great for visualization!

44. What is overfitting and how can we prevent it?

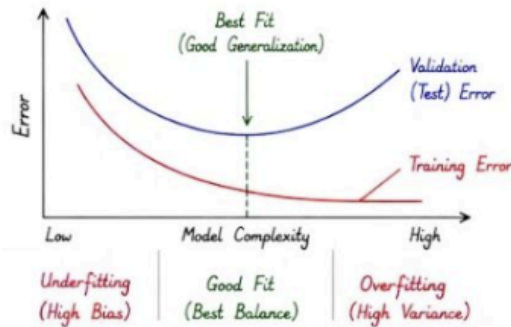
Overfitting happens when a model learns the training data too well, including the noise, and performs poorly on unseen data.

Signs of Overfitting:

- Very low training error.
- High validation/test error.

How to prevent?

- Use more training data.
- Simplify the model.
- Apply regularization (L1/L2).
- Use dropout (for neural networks).
- Early stopping.
- Cross-validation.



Exam Tip

Overfitting = low training error but high test error.
Goal is good generalization, not perfect fit.

45. What is a confusion matrix and why is it important?

A confusion matrix is a table used to evaluate the performance of a classification model by comparing predicted labels with actual labels.

Confusion Matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

Why important?

- Gives a detailed picture of model performance.
- Helps when classes are imbalanced.
- Shows types of errors the model is making.
- Basis for metrics like Accuracy, Precision, Recall, F1 Score, etc.

Exam Tip

Confusion matrix helps you go beyond accuracy and understand the real performance.

46. What is the difference between bagging and boosting?

Both are ensemble techniques but differ in approach.

Aspect	Bagging	Boosting
Full Form	Bootstrap Aggregating	Boosting
Approach	Parallel (Independent models)	Sequential (Models depend on previous ones)
Training Data	Uses random subsets (with replacement) of data	Uses same data, but focuses more on misclassified samples
Combines	Average (for regression) or Majority vote (for classification)	Weighted combination of models
Examples	Random Forest, Bagged Trees	AdaBoost, Gradient Boosting, XGBoost
Effect	Reduces variance	Reduces bias (and variance)

Exam Tip

Bagging builds many independent models.
Boosting builds models that learn from previous mistakes.

47. What is gradient descent?

Gradient Descent is an optimization algorithm used to minimize the loss function by iteratively moving in the direction of the negative gradient.

How it works:

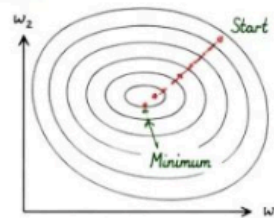
- 1 Start with initial weights (random).
- 2 Compute the gradient of the loss.
- 3 Update weights in the opposite direction of the gradient.
- 4 Repeat until convergence.

Update Rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla L(w)$$

where:

- w = weights
- η = learning rate
- $\nabla L(w)$ = gradient of loss



Exam Tip

Learning rate (η) is critical.
Too high $\rightarrow$ overshoot.
Too low $\rightarrow$ slow learning.
Just right $\rightarrow$ fast convergence.

Types of Gradient Descent:

- Batch GD: Uses all data
- Stochastic GD: Uses one sample at a time
- Mini-batch GD: Uses small batches

48. What is dimensionality reduction and why is it used?

Dimensionality reduction is the process of reducing the number of input features in a dataset while preserving as much important information as possible.

Why is it used?

- Reduces computational cost.
- Helps remove noise and redundancy.
- Mitigates curse of dimensionality.
- Improves model performance (less overfitting).
- Helps in visualization.

Common Techniques:

Technique	Type	Key Idea
PCA	Linear	Projects data onto principal components (max variance).
SVD	Linear	Factorizes the matrix into U, Σ, V^T and keeps top components.
t-SNE	Non-linear	Preserves local structure in lower dimensions.
UMAP	Non-linear	Preserves local and global structure efficiently.

Exam Tip

Use PCA for speed and interpretability.
Use t-SNE/UMAP for visualization of complex, non-linear data.

49. What is the bias-variance tradeoff with a graph?

The bias-variance tradeoff is the balance between bias (error from wrong assumptions) and variance (error from sensitivity to training data).

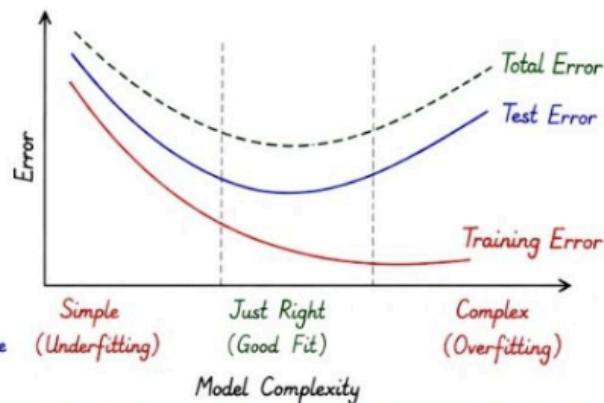
Bias (High bias):

- Model is too simple.
- Underfits the data.
- High training error and high test error.

Variance (High variance):

- Model is too complex.
- Overfits the data.
- Low training error but high test error.

Goal: Find the right balance to minimize total error.



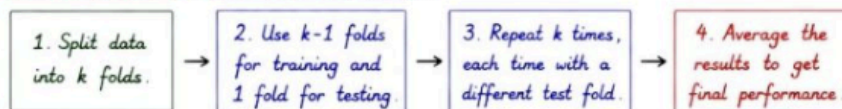
Exam Tip

High bias → underfitting.
High variance → overfitting.
Best model is in the "just right" middle zone.

50. What is cross-validation and what are its types?

Cross-validation is a technique to evaluate a model by splitting the data into multiple folds and training/testing the model multiple times.

How it works? (k-Fold Cross-Validation)



Types of Cross-Validation:

- Hold-Out CV : Single split (e.g., 80% train, 20% test). Simple but high variance.
- K-Fold CV : Data split into k folds. More stable and widely used.
- Stratified K-Fold : Preserves class distribution in each fold. Good for imbalanced data.
- Leave-One-Out CV (LOOCV): $k = n$ (number of samples). Low bias, high variance, expensive.
- Time Series CV : Respects time order. Train on past, test on future.

Benefits:

- Makes efficient use of data.
- Helps in model selection.
- Reduces variance in evaluation.
- Detects overfitting early.

Exam Tip

Cross-validation gives a more reliable estimate of model performance compared to a single train/test split.

This is often the best starting point for enterprise RAG because semantic search understands meaning while keyword search catches exact IDs, names, codes and terminology.

Trade-off: More infrastructure and retrieval complexity.

- ♦ 2. GraphRAG

Useful when the answer depends on relationships between entities, not just similar text.

Think: people → companies → projects → technologies → locations.

Trade-off: Building and maintaining entity relationships can become expensive and complicated.

- ♦ 3. Agentic RAG

Retrieval becomes an iterative process.

The agent can decide to:

- Search the vector database
- Query SQL
- Search the web
- Evaluate the results
- Retrieve again if necessary

Trade-off: More reasoning means more latency, cost and failure modes.

- ♦ 4. Corrective RAG (CRAG)

Don't blindly trust retrieved documents.

A grader evaluates the retrieved context first. If it's weak, the system can rewrite the query, retrieve again or fall back to another source.

Trade-off: Additional evaluation and retrieval steps increase complexity and latency.

- ♦ 5. Multimodal RAG

Because enterprise knowledge isn't just text.

PDFs contain tables.

Reports contain charts.

Documents contain images and scanned forms.

Multimodal RAG allows these different information types to participate in retrieval.

Trade-off: Multimodal ingestion, embeddings and evaluation are significantly harder.

The important part?

You don't need all five.

In a real production system, I would usually start simple:

Hybrid RAG → add corrective retrieval where accuracy matters → introduce agentic retrieval only where the problem actually requires it.

Every additional component introduces another:

latency + cost + failure mode.

That's why good AI architecture isn't about adding more agents, more tools or more sophisticated RAG patterns.

It's about knowing when NOT to add them.

The best AI engineers don't build the most complicated architecture.